VIT BHOPAL UNIVERSITY

Project Report on

“EXPENSE TRACKER”

B.Tech First Year

(Branch – Computer Science and Engineering)

Submitted To:

Dr. M. K. Jayanti

Submitted By:

Anukreeti Singh

25BCE11154

ACKNOWLEDGEMENT

This Project report was completed as a result of support from many people, although not all of them can be mentioned.

We wish to express our sincere gratitude to God for his protection, providence, guidance and above all, for sustaining us.

We are greatly indebted to our good supervisor Dr. M. K. Jayanti Ma'am for her useful and necessary observation, suggestions, contribution and corrections. We would not have been able to achieve anything in this research without your supervision. May God enrich you greatly in every area of life.

Finally, we wish to express our appreciation to our parents for their love and support.

Anukreeti Singh

1 Introduction

In today's fast-paced student life, managing personal finances often becomes an overlooked responsibility. Students frequently engage in small, everyday transactions such as buying food, using transportation, or spending on entertainment. While each expense may seem insignificant individually, their cumulative effect can lead to overspending.

This project focuses on developing an AI-based Expense Tracker that simplifies financial management for students by automating expense categorization, tracking spending patterns, and providing timely alerts when budget limits are approached or exceeded.

The goal is to combine simplicity, automation, and intelligence into a single system that encourages better financial habits.

2 Problem Statement

Students often fail to keep track of their daily expenses. Small purchases such as food, travel, or entertainment accumulate over time, leading to overspending. Most students only realize they have exceeded their budget after it is too late, which results in regret and poor financial habits.

Existing solutions are either too complex, require manual categorization, or do not provide real-time feedback. Therefore, there is a need for a simple, intelligent system that can automatically track expenses and alert users when they are nearing or exceeding their budget.

3 Motivation

The motivation behind this project is deeply personal and relatable. As a student, I have experienced situations where I spent money casually without tracking it, only to realize later that I had exceeded my budget. This is a very common issue among students — we tend to ignore small expenses, assuming they don't matter, but they add up quickly.

What makes it worse is that by the time we realize it, the money is already spent and all we can do is regret it.

This project was built to solve exactly that problem by:

- Making expense tracking effortless
- Providing instant alerts
- Helping users stay aware of their financial limits

4 Objective

The main objectives of this project are:

- To create a system for recording daily expenses
- To automate expense categorization using Machine Learning
- To track and compare total spending against a predefined budget
- To provide real-time alerts when the budget is nearing or exceeded
- To visualize spending patterns for better understanding
- To build a simple and user-friendly interface

5 Methodology, Implementation and Rationale

The project is implemented using Python and integrates multiple components. This section explains how the system was implemented and the reasoning behind the key technical choices made during development.

1. <u>Programming Language and Libraries</u>

The entire system was implemented using Python, due to its simplicity, readability, and strong ecosystem of libraries.

The following libraries were used:

- pandas → for data handling and analysis
- matplotlib → for data visualization
- scikit-learn → for Machine Learning models
- tkinter → for building the graphical user interface
- csv → for file storage

Rationale: Python was chosen because it allows rapid development and easy integration of Machine Learning and GUI components in a single application.

2. <u>Data Storage Approach</u>

The application stores all expenses in a CSV file (expenses.csv), which contains:

- Date
- Description
- Category
- Amount

Rationale: Instead of using a database, CSV was selected because:

- It is simple and lightweight
- Easy to read and modify
- Suitable for small-scale applications like this project

This decision keeps the system beginner-friendly while still functional.

3. Expense Categorization using Machine Learning

The system uses a Multinomial Naive Bayes classifier to categorize expenses automatically based on text descriptions.

- CountVectorizer converts text input into numerical vectors
- MultinomialNB is trained on predefined examples
- Categories include:
    - Food
    - Travel
    - Entertainment
    - Stationary
    - Fashion
    - Essentials

Rationale: Naive Bayes was chosen because:

- It is fast and efficient
- Works well for text classification problems
- Requires less computational power
- Easy to implement and understand

4. Self-Learning Mechanism

A key feature of the system is its ability to improve over time.

- The model calculates prediction confidence
- If confidence is low:
    - The user is prompted to enter the correct category
    - The new data is added to the training dataset
    - The model is retrained immediately

Rationale: This design allows the system to:

- Adapt to new types of inputs
- Improve accuracy gradually
- Reduce dependency on predefined data

This transforms the system from a static model into a dynamic learning system.

5. Budget Monitoring System

The application continuously tracks total expenses and compares them with the user-defined budget.

Three levels of alerts are implemented:

- 80% of budget used → Warning
- 100% reached → Notification
- Exceeded → Critical alert

These alerts are displayed using popup messages in the GUI.

Rationale: Instead of only tracking expenses, the system actively prevents overspending by:

- Notifying users early
- Encouraging timely decisions
- Reducing financial regret

This is one of the most important features of the project.

6. Graphical User Interface (GUI)

The user interface is built using Tkinter.

The GUI includes:

- Input fields for:
    - Budget
    - Description
    - Amount
- Buttons for:
    - Adding expenses
    - Viewing summary
    - Checking budget status
    - Predicting spending
- Popup messages for:

    - Alerts
    - Errors
    - Confirmations

Rationale: Tkinter was chosen because:

- It is built into Python (no extra installation required)
- Simple to use
- Suitable for small desktop applications

The GUI replaces command-line interaction, making the application more user-friendly.

7. Data Visualization

The system uses bar charts to display category-wise spending.:

- Data is grouped by category using pandas
- matplotlib is used to generate bar graphs

Rationale:Visualization helps users:

- Quickly understand spending patterns
- Identify high-expense categories
- Make better financial decisions

8. Expense Prediction

A Linear Regression model is used to predict future expenses.

- Each expense is assigned a sequential day value
- The model learns the trend between time and spending
- Predicts the next expected expense

Rationale: Although simple, this feature introduces predictive analytics and demonstrates:

- How past data can influence future estimates
- Practical use of regression models

9. Error Handling and Validation

Basic error handling is implemented throughout the system.

- Try-except blocks prevent crashes
- Input validation ensures:
    - Amount is numeric
    - Budget is properly set

Rationale: This ensures:

- Smooth user experience
- Prevention of unexpected program failures

10. Execution Environment Consideration

During development, it was observed that Tkinter does not function properly in Jupyter Notebook environments.

- The application is designed to run as a .py script in a local Python environment

Rationale: This ensures:

- Proper GUI rendering
- Stable execution

Summary of Design Philosophy

The overall implementation focuses on:

- Simplicity → Easy to use and understand
- Automation → Minimal manual effort
- Adaptability → Learns from user input
- Practicality → Solves a real student problem

Each technical decision was made to balance functionality with ease of implementation.

6 Key Design Decisions

Several important decisions were made during development:

- Using CSV instead of a database → Keeps the project simple and easy to manage
- Choosing Naive Bayes → Lightweight and suitable for text classification
- Implementing self-learning → Makes the system adaptive rather than static
- Adding real-time alerts → Prevents regret instead of just tracking history

Switching from CLI to GUI → Improves usability significantly

7 Challenges Faced

This project involved multiple challenges:

- Integrating Machine Learning with a GUI application
- Handling low-confidence predictions without interrupting user experience
- Debugging issues caused by running Tkinter in Jupyter Notebook
- Ensuring the application runs correctly in a proper Python environment
- Designing a system that is both simple and functional
- Managing silent errors due to exception handling

These challenges required iterative debugging and understanding of both system behavior and environment limitations.

8 Results

The final system successfully:

- Records and stores expenses efficiently
- Automatically categorizes most inputs correctly
- Learns from user corrections over time
- Provides real-time budget alerts
- Displays summaries and visualizations
- Predicts future expenses

The application is functional, interactive, and aligned with the original objectives.

9 Learning Outcomes

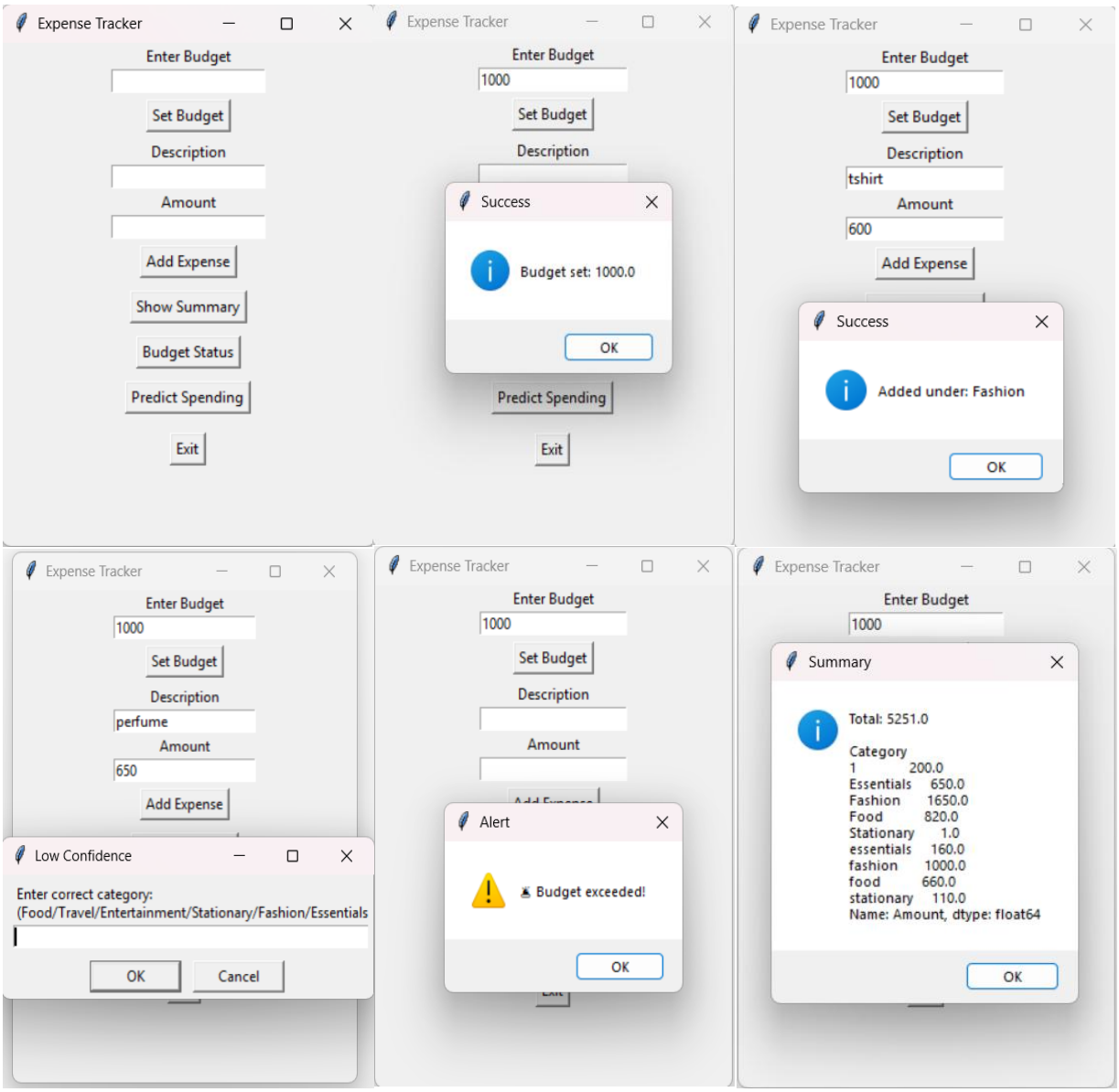
This project helped me understand:

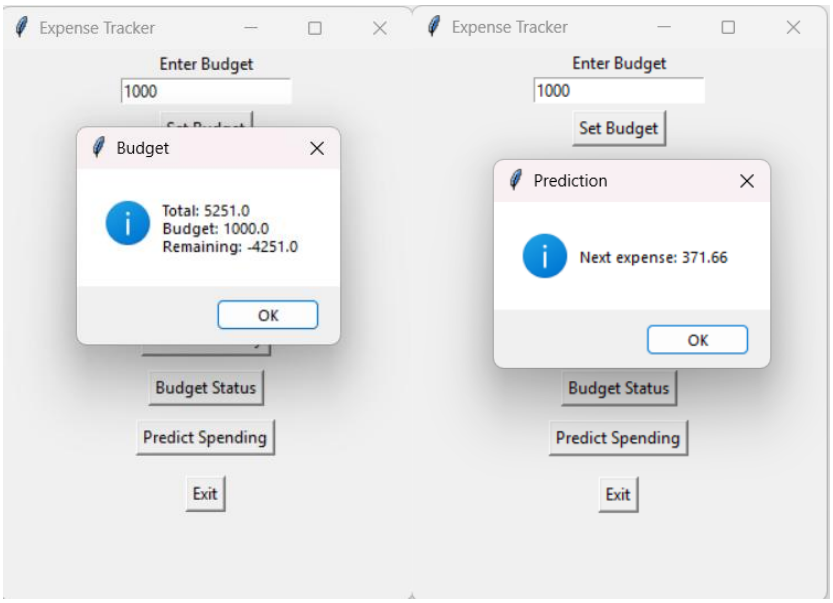
- How Machine Learning can be applied to real-world problems
- Basics of text classification using Naive Bayes

- GUI development using Tkinter
- Handling file operations and data processing using pandas
- The importance of choosing the right development environment
- Debugging practical issues beyond theoretical knowledge

Most importantly, I learned how to convert an everyday problem into a working technical solution.

10 Screenshots of the Result





11 Future Improvements

The project can be enhanced further by:

- Designing a modern UI using advanced frameworks
- Adding user authentication (login system)
- Storing data in a database instead of CSV
- Improving prediction models
- Converting the application into a standalone executable
- Adding mobile or web-based access

12 Conclusion

This project demonstrates that even simple AI techniques can be powerful when applied to everyday problems. By combining automation, user interaction, and real-time feedback, the system helps students stay aware of their financial habits and avoid unnecessary overspending.

It not only solves a practical issue but also promotes better decision-making and responsibility in managing personal finances.

13 References

1. Python Programming Language Documentation: https://docs.python.org/3/
2. pandas Documentation: https://pandas.pydata.org/docs/
3. Matplotlib Documentation: https://matplotlib.org/stable/contents.html
4. scikit-learn Documentation: https://scikit-learn.org/stable/
5. Naive Bayes classifier: https://en.wikipedia.org/wiki/Naive_Bayes_classifier
6. Linear Regression: https://en.wikipedia.org/wiki/Linear_regression
7. Tkinter Documentation: https://docs.python.org/3/library/tkinter.html
8. CountVectorizer: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html
9. Multinomial Naive Bayes: https://scikit-learn.org/stable/modules/naive_bayes.html